Run samples of the ServiceQueue simulation Aylene

Collect statistics and plot histograms along the way.

Set up

We'll measure time in hours

Arrival rate: 10 per hour

```
lambda = 10;
```

Departure (service) rate: 1 per 5 minutes, so 12 per hour

```
mu = 12;
```

Number of serving stations

```
s = 1;
```

Run 100 samples of the queue.

```
NumSamples = 100;
```

Each sample is run up to a maximum time.

```
MaxTime = 96;
```

Make a log entry every so often

```
LogInterval = 1/60;
```

Numbers from theory for M/M/1 queue

Compute `P(1+n)` = P_n = probability of finding the system in state n in the long term. Note that this calculation assumes $s = 1$.

```
rho = lambda / mu;
P0 = 1 - rho;
nMax = 10;
P = zeros([1, nMax+1]);
P(1) = P0;
for n = 1:nMax
    P(1+n) = P0 * rho^n;
end
```

Run simulation samples

This is the most time consuming calculation in the script, so let's put it in its own section. That way, we can run it once, and more easily run the faster calculations multiple times as we add features to this script.

Reset the random number generator. This causes MATLAB to use the same sequence of pseudo-random numbers each time you run the script, which means the results come out exactly the same. This is a good idea for testing purposes. Under other circumstances, you probably want the random numbers to be truly unpredictable and you wouldn't do this.

```
rng("default");
```

We'll store our queue simulation objects in this list.

```
QSamples = cell([NumSamples, 1]);
```

The statistics come out weird if the log interval is too short, because the log entries are not independent enough. So the log interval should be long enough for several arrival and departure events happen.

```
for SampleNum = 1:NumSamples
    fprintf("Working on sample %d\n", SampleNum);
    q = ServiceQueue( ...
        ArrivalRate=lambda, ...
        DepartureRate=mu, ...
        NumServers=s, ...
        LogInterval=LogInterval);
    q.schedule_event(Arrival(random(q.InterArrivalDist), Customer(1)));
    run_until(q, MaxTime);
    QSamples{SampleNum} = q;
end
```

```
Working on sample 1
Working on sample 2
Working on sample 3
Working on sample 4
Working on sample 5
Working on sample 6
Working on sample 7
Working on sample 8
Working on sample 9
Working on sample 10
Working on sample 11
Working on sample 12
Working on sample 13
Working on sample 14
Working on sample 15
Working on sample 16
Working on sample 17
Working on sample 18
Working on sample 19
Working on sample 20
Working on sample 21
Working on sample 22
Working on sample 23
Working on sample 24
Working on sample 25
Working on sample 26
Working on sample 27
Working on sample 28
Working on sample 29
Working on sample 30
Working on sample 31
Working on sample 32
Working on sample 33
```

```
Working on sample 34
Working on sample 35
Working on sample 36
Working on sample 37
Working on sample 38
Working on sample 39
Working on sample 40
Working on sample 41
Working on sample 42
Working on sample 43
Working on sample 44
Working on sample 45
Working on sample 46
Working on sample 47
Working on sample 48
Working on sample 49
Working on sample 50
Working on sample 51
Working on sample 52
Working on sample 53
Working on sample 54
Working on sample 55
Working on sample 56
Working on sample 57
Working on sample 58
Working on sample 59
Working on sample 60
Working on sample 61
Working on sample 62
Working on sample 63
Working on sample 64
Working on sample 65
Working on sample 66
Working on sample 67
Working on sample 68
Working on sample 69
Working on sample 70
Working on sample 71
Working on sample 72
Working on sample 73
Working on sample 74
Working on sample 75
Working on sample 76
Working on sample 77
Working on sample 78
Working on sample 79
Working on sample 80
Working on sample 81
Working on sample 82
Working on sample 83
Working on sample 84
Working on sample 85
Working on sample 86
Working on sample 87
Working on sample 88
Working on sample 89
Working on sample 90
Working on sample 91
Working on sample 92
Working on sample 93
Working on sample 94
Working on sample 95
Working on sample 96
Working on sample 97
Working on sample 98
```

```
Working on sample 99
Working on sample 100
```

Collect measurements of how many customers are in the system

Count how many customers are in the system at each log entry for each sample run. There are two ways to do this. You only have to do one of them.

Option one: Use a for loop.

```
NumInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    % Pull out samples of the number of customers in the queue system.
Each
    % sample run of the queue results in a column of samples of customer
    % counts, because tables like q.Log allow easy extraction of whole
    % columns like this.
    NumInSystemSamples{SampleNum} = q.Log.NumWaiting +
q.Log.NumInService;
end
```

Option two: Map a function over the cell array of ServiceQueue objects.

The `@(q) ...` expression is shorthand for a function that takes a `ServiceQueue` as input, names it `q`, and computes the sum of two columns from its log. The `cellfun` function applies that function to each item in `QSamples`. The option `UniformOutput=false` tells `cellfun` to produce a cell array rather than a numerical array.

```
NumInSystemSamples = cellfun( ...
    @(q) q.Log.NumWaiting + q.Log.NumInService, ...
    QSamples, ...
    UniformOutput=false);
```

Join numbers from all sample runs

.

`vertcat` is short for "vertical concatenate", meaning it joins a bunch of arrays vertically, which in this case results in one tall column.

```
NumInSystem = vertcat(NumInSystemSamples{:});
```

MATLAB-ism: When you pull multiple items from a cell array, the result is a "comma-separated list" rather than some kind of array. Thus, the above means

`NumInSystem = vertcat(NumInSystemSamples{1}, NumInSystemSamples{2}, ...)`

which concatenates all the columns of numbers in NumInSystemSamples into one long column.

This is roughly equivalent to "splatting" in Python, which looks like `f(*args)`.

```
% Option 1 — For loop (your code)
NumInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    NumInSystemSamples{SampleNum} = q.Log.NumWaiting +
q.Log.NumInService;
end
```

```
assert(iscell(NumInSystemSamples) && numel(NumInSystemSamples)>0, ...
    'NumInSystemSamples must be a nonempty cell array.');
```

Pictures and stats for number of customers in system

Print out mean number of customers in the system.

```
meanNumInSystem = mean(NumInSystem);
fprintf("Mean number in system: %f\n", meanNumInSystem);
```

```
Mean number in system: 4.808672
```

Make a figure with one set of axes.

```
fig = figure();
t = tiledlayout(fig,1,1);
ax = nexttile(t);
```

MATLAB-ism: Once you've created a picture, you can use `hold` to cause further plotting functions to work with the same picture rather than create a new one.

```
hold(ax, "on");
```

Start with a histogram. The result is an empirical PDF, that is, the area of the bar at horizontal index n is proportional to the fraction of samples for which there were n customers in the system. The data for this histogram is counts of customers, which must all be whole numbers. The option `BinMethod="integers"` means to use bins $(-0.5, 0.5), (0.5, 1.5), \ldots$ so that the height of the first bar is proportional to the count of 0s in the data, the height of the second bar is proportional to the count of 1s, etc. MATLAB can choose bins automatically, but since we know the data consists of whole numbers, it makes sense to specify this option so we get consistent results.

```
h = histogram(ax, NumInSystem, Normalization="probability",
BinMethod="integers");
```

Plot $(0, P_0), (1, P_1), \ldots$. If all goes well, these dots should land close to the tops of the bars of the histogram.

```
plot(ax, 0:nMax, P, 'o', MarkerEdgeColor='k', MarkerFaceColor='r');
```

Add titles and labels and such.

```
title(ax, "Number of customers in the system");
xlabel(ax, "Count");
ylabel(ax, "Probability");
legend(ax, "simulation", "theory");
```

Set ranges on the axes. MATLAB's plotting functions do this automatically, but when you need to compare two sets of data, it's a good idea to use the same ranges on the two pictures. To start, you can let MATLAB choose the ranges automatically, and just know that it might choose very different ranges for different sets of data. Once you're certain the picture content is correct, choose an x range and a y range that gives good results for all sets of data. The final choice of ranges is a matter of some trial and error. You generally have to do these commands *after* calling `plot` and `histogram`.

This sets the vertical axis to go from 0 to 0.3.

```
ylim(ax, [0, 0.3]);
```

This sets the horizontal axis to go from -1 to 21. The histogram will use bins $(-0.5, 0.5), (0.5, 1.5), \ldots$ so this leaves some visual breathing room on the left.

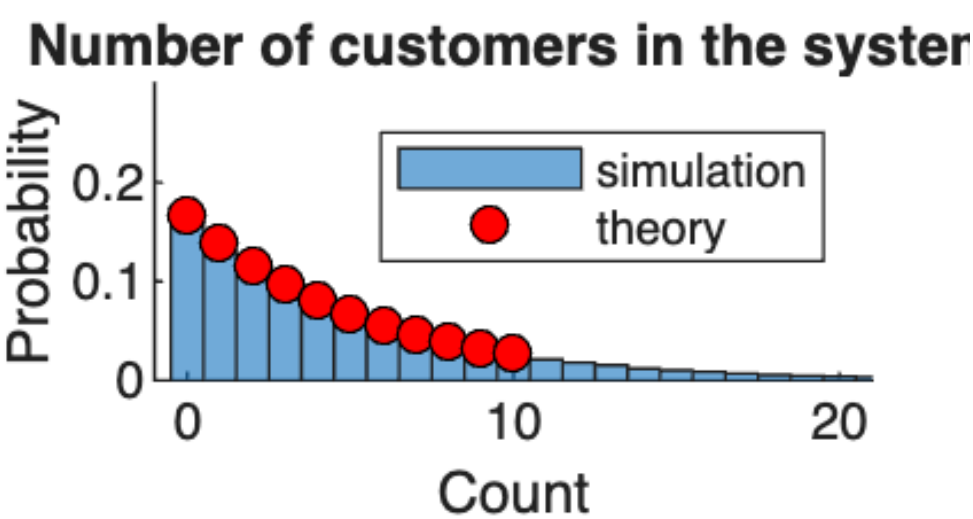
```
xlim(ax, [-1, 21]);
```

MATLAB-ism: You have to wait a couple of seconds for those settings to take effect or `exportgraphics` will screw up the margins.

```
pause(2);
```

Save the picture as a PDF file.

```
exportgraphics(fig, "Number in system histogram.pdf");
```



Collect measurements of how long customers spend in the system

This is a rather different calculation because instead of looking at log entries for each sample `ServiceQueue`, we'll look at the list of served customers in each sample `ServiceQueue`.

Option one: Use a for loop.

```
TimeInSystemSamples = cell([NumSamples, 1]);
for SampleNum = 1:NumSamples
    q = QSamples{SampleNum};
    % The next command has many parts.
    %
    % q.Served is a row vector of all customers served in this particular
    % sample.
    % The ' on q.Served' transposes it to a column.
    %
    % The @(c) ... expression below says given a customer c, compute its
    % departure time minus its arrival time, which is how long c spent in
    % the system.
    %
    % cellfun(@(c) ..., q.Served') means to compute the time each
customer
    % in q.Served spent in the system, and build a column vector of the
    % results.
    %
    % The column vector is stored in TimeInSystemSamples{SampleNum}.
    TimeInSystemSamples{SampleNum} = ...
        cellfun(@(c) c.DepartureTime - c.ArrivalTime, q.Served');
end
```

Option two: Use `cellfun` twice.

The outer call to `cellfun` means do something to each `ServiceQueue` object in `QSamples`. The "something" it does is to look at each customer in the `ServiceQueue` object's list `q.Served` and compute the time it spent in the system.

```
TimeInSystemSamples = cellfun( ...
    @(q) cellfun(@(c) c.DepartureTime - c.ArrivalTime, q.Served'), ...
    QSamples, ...
    UniformOutput=false);
```

Join them all into one big column.

```
TimeInSystem = vertcat(TimeInSystemSamples{:});
```

Pictures and stats for time customers spend in the system

Print out mean time spent in the system.

```
meanTimeInSystem = mean(TimeInSystem);
fprintf("Mean time in system: %f\n", meanTimeInSystem);
```

```
Mean time in system: 0.480263
```

Make a figure with one set of axes.

```
fig = figure();
t = tiledlayout(fig,1,1);
ax = nexttile(t);
```

This time, the data is a list of real numbers, not integers. The option `BinWidth=...` means to use bins of a particular width, and choose the left-most and right-most edges automatically. Instead, you could specify the left-most and right-most edges explicitly. For instance, using `BinEdges=0:0.5:60` means to use bins $(0, 0.5), (0.5, 1.0), \ldots$

```
h = histogram(ax, TimeInSystem, Normalization="probability",
BinWidth=5/60);
```

Add titles and labels and such.

```
title(ax, "Time in the system");
xlabel(ax, "Time");
ylabel(ax, "Probability");
```

Set ranges on the axes.

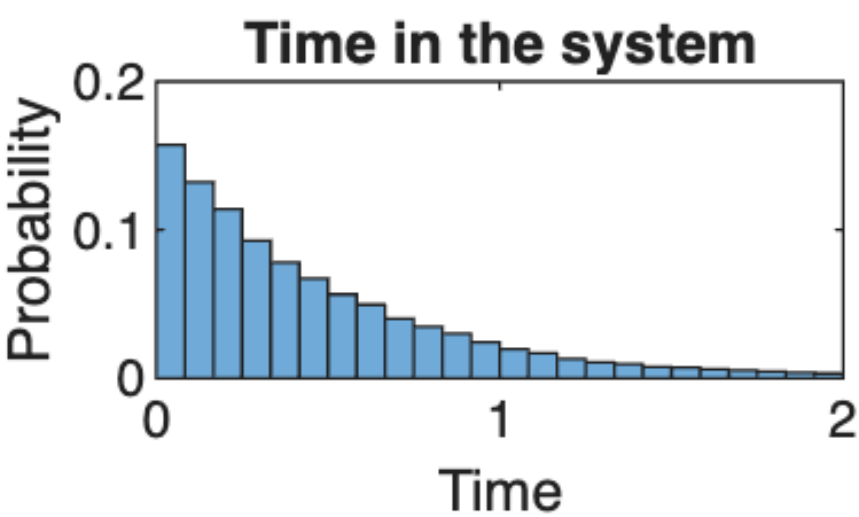
```
ylim(ax, [0, 0.2]);
xlim(ax, [0, 2.0]);
```

Wait for MATLAB to catch up.

```
pause(2);
```

Save the picture as a PDF file.

```
exportgraphics(fig, "Time in system histogram.pdf");
```



```
% Example scalars
theory_mean = 2.5;
sim_mean    = 2.65;

% Example vectors (multiple statistics)
theory = [theory_mean, 0.1, 5];    % replace with your theoretical values
sim    = [sim_mean,    0.09, 4.8]; % replace with your simulated
estimates

% Safe percent discrepancy calculation
pct = nan(size(theory));                % preallocate
nonzero = theory ~= 0;                  % indices where theory nonzero
pct(nonzero) = 100 * abs(sim(nonzero) - theory(nonzero)) ./
abs(theory(nonzero));
pct(~nonzero)  = abs(sim(~nonzero) - theory(~nonzero)); % absolute diff
when theory == 0

% Display nicely
for k = 1:numel(theory)
    if theory(k) ~= 0
        fprintf('Stat %d: theory = %.4g, sim = %.4g, discrepancy = %.3f%
%n', ...
                k, theory(k), sim(k), pct(k));
    else
        fprintf('Stat %d: theory = 0, sim = %.4g, abs discrepancy =
%.4g\n', ...
                k, sim(k), pct(k));
    end
end
```

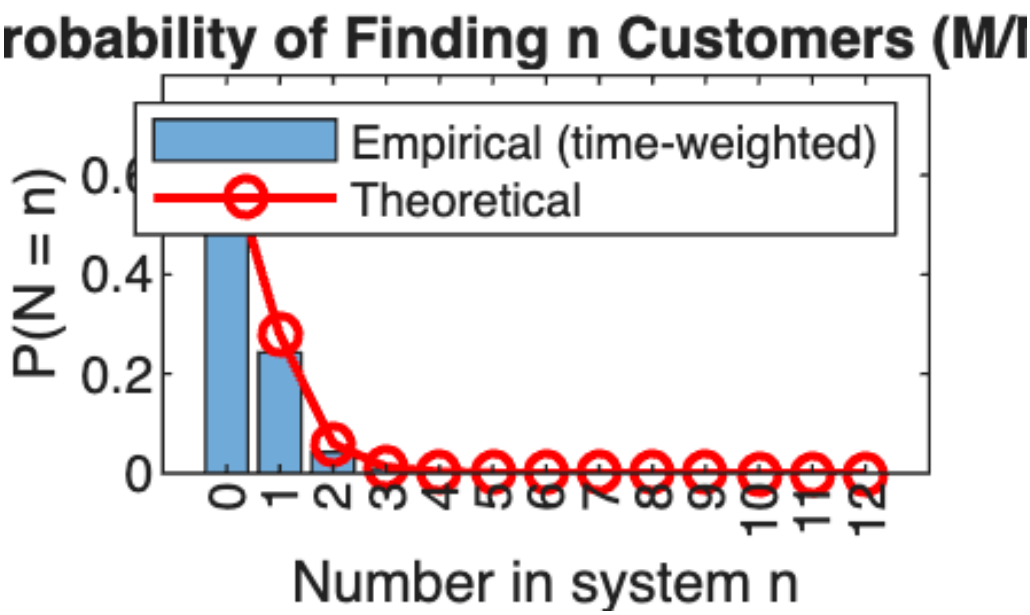
```
Stat 1: theory = 2.5, sim = 2.65, discrepancy = 6.000%nStat 2: theory = 0.1, sim = 0.09, discrepancy =
```

```
% M/M/2 next-event simulation for 8 hours
rng(1) % for reproducibility

% Parameters (hours)
c = 2;
lambda = 13.3333;    % arrivals per hour
mu = 31.033;         % service rate per server per hour
T_end = 8;           % hours

% State and record arrays
t = 0;
num_in_system = 0;
server_busy = zeros(1,c);        % 0 free, 1 busy
service_end = inf(1,c);          % next completion times
next_arrival = exprnd(1/lambda);

% Records
times_record = [];               % times at which state changes
```

```
n_record = [];                    % num_in_system at those times
sojourns = [];                    % completed customers' sojourn times
arrival_times = [];               % arrival time for each customer in
queue
queue_arrival_times = {};         % arrival times waiting when servers
busy

% Initialize
times_record(end+1) = t;
n_record(end+1) = num_in_system;
queue_arrival_times = {};

while t < T_end
    % find next event
    [t_next_srv, idx_srv] = min(service_end);
    if next_arrival <= t_next_srv && next_arrival <= T_end
        t = next_arrival;
        % Arrival event
        num_in_system = num_in_system + 1;
        times_record(end+1) = t;
        n_record(end+1) = num_in_system;
        % find a free server
        free_idx = find(server_busy==0,1);
        if ~isempty(free_idx)
            % start service immediately
            server_busy(free_idx) = 1;
            service_end(free_idx) = t + exprnd(1/mu);
            % record arrival time to compute sojourn later: store with
server index
            arrival_times = [arrival_times; t, free_idx]; %#ok<AGROW>
        else
            % join queue (store arrival time)
            queue_arrival_times{end+1} = t;
        end
        % schedule next arrival
        next_arrival = t + exprnd(1/lambda);
    else
        % Service completion event (or no more arrivals before T_end)
        if t_next_srv > T_end
            break;
        end
        t = t_next_srv;
        % process completion at idx_srv
        num_in_system = num_in_system - 1;
        times_record(end+1) = t;
        n_record(end+1) = num_in_system;
        % find corresponding arrival time for this completion
        % We recorded arrival_times rows [arrival_time, server_idx]
        % find the matching server entry (most recent with that server)
        idx_match = find(arrival_times(:,2) == idx_srv,1,'last');
        if ~isempty(idx_match)
            sojourns(end+1) = t - arrival_times(idx_match,1); %#ok<AGROW>
```

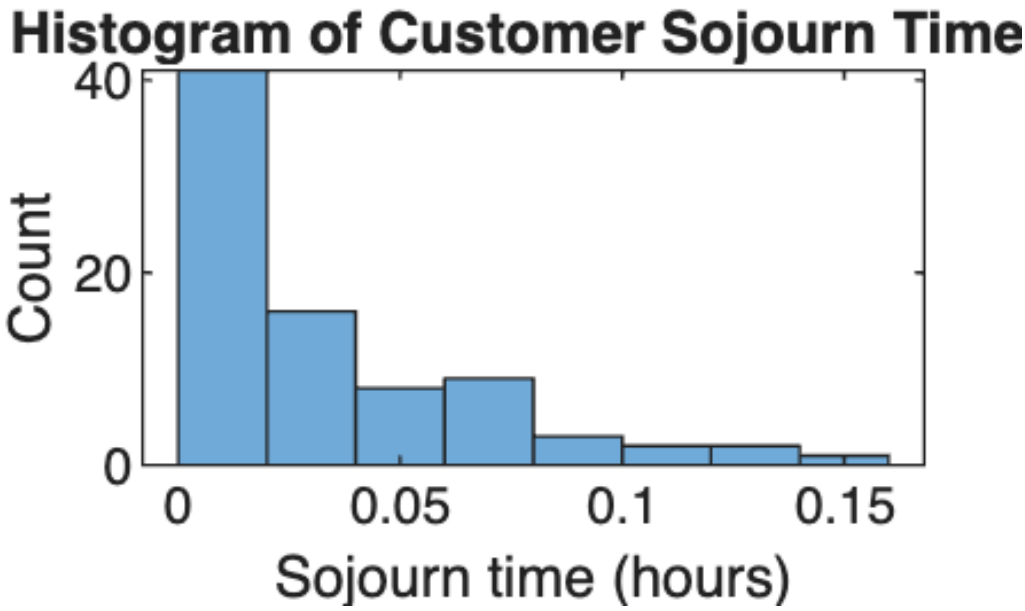
```
            arrival_times(idx_match,:) = []; % remove record
        end
        % free server
        server_busy(idx_srv) = 0;
        service_end(idx_srv) = inf;
        % if queue not empty, start next customer
        if ~isempty(queue_arrival_times)
            arr_t = queue_arrival_times{1};
            queue_arrival_times(1) = [];
            server_busy(idx_srv) = 1;
            service_end(idx_srv) = t + exprnd(1/mu);
            % record start: arrival time saved to compute sojourn
            arrival_times = [arrival_times; arr_t, idx_srv]; %#ok<AGROW>
        end
    end
end

% If simulation ended while some services still running, let them finish
(optional)
% Here we just ignore completions after T_end for sojourn histogram,
% or we could run until system empties. We'll keep sojourns collected
only up to T_end.

% Build empirical distribution of N over time (time-weighted)
% times_record and n_record give piecewise-constant N(t). Compute total
time spent in each n.
edges_t = times_record;
durations = diff([edges_t, T_end]); % last interval truncated at T_end
Ns = n_record;
maxN = max(Ns);
time_in_state = zeros(1,maxN+10);
for k=1:length(Ns)
    n = Ns(k);
    time_in_state(n+1) = time_in_state(n+1) + durations(k);
end
prob_empirical = time_in_state / T_end;
n_vals = 0:(length(prob_empirical)-1);

% Theoretical P_n for M/M/c (use your formula)
rho = lambda / (mu);
c_rho = rho / c; % = ρ/c? careful: define traffic per server as lambda/
(c*mu)
% We'll compute with standard formula using offered load a = lambda/mu
a = lambda / mu;
P0_sum = 0;
for n=0:(c-1)
    P0_sum = P0_sum + a^n / factorial(n);
end
term = a^c / factorial(c) * (1 / (1 - a/(c)));
P0 = 1 / (P0_sum + term);
P_theoretical = zeros(1,100);
for n=0:99
```

```
    if n < c
        P_theoretical(n+1) = (a^n / factorial(n)) * P0;
    else
        P_theoretical(n+1) = (a^n / (factorial(c) * c^(n-c))) * P0;
    end
end
% Trim theoretical vector to match empirical max
P_theo_trim = P_theoretical(1:min(length(P_theoretical),
length(prob_empirical)));

% Plot probability of finding n customers: empirical vs theoretical
figure;
bar(n_vals, prob_empirical(1:length(n_vals)), 'FaceAlpha',0.6); hold on
plot(0:(length(P_theo_trim)-1), P_theo_trim, 'r-o','LineWidth',1.5);
xlabel('Number in system n');
ylabel('P(N = n)');
legend('Empirical (time-weighted)','Theoretical');
title('Probability of Finding n Customers (M/M/2)');
```




```
% Histogram of sojourn times (hours) and some stats
figure;
if isempty(sojourns)
    warning('No completed customers before T_end. Increase T_end or
allow completions after T_end.');
else
    histogram(sojourns, 'BinMethod','fd');
    xlabel('Sojourn time (hours)');
    ylabel('Count');
    title('Histogram of Customer Sojourn Times');
    fprintf('Sojourn count: %d\nMean sojourn (empirical): %.6f h\nStd
sojourn: %.6f h\n', ...
        length(sojourns), mean(sojourns), std(sojourns));
    % Compare to theoretical W = L / lambda
    W_theo = 0.06; % your given
```

```
    fprintf('Theoretical mean sojourn W = %.6f h\n', W_theo);
end
```




```
Sojourn count: 82
Mean sojourn (empirical): 0.033684 h
Std sojourn: 0.033949 h
Theoretical mean sojourn W = 0.060000 h
```

```
% --- Inputs from your simulation ---
% prob_empirical: time-weighted P(N=n) vector (indexed n=0..)
% sojourns: vector of completed customer sojourn times (hours)
% lambda, mu, c defined as in simulation
% T_end (simulation length) defined

% Example names assumed present from prior code:
% prob_empirical, n_vals, lambda, mu, c, sojourns

% Compute theoretical P_n (up to same length as empirical)
a = lambda / mu;                        % offered load
c_val = c;
% compute P0
P0_sum = 0;
for n=0:(c_val-1)
    P0_sum = P0_sum + a^n / factorial(n);
end
term = a^c_val / factorial(c_val) * (1 / (1 - a/c_val));
P0 = 1 / (P0_sum + term);

Nmax = length(prob_empirical)-1;
P_theo = zeros(1,Nmax+1);
for n=0:Nmax
    if n < c_val
        P_theo(n+1) = (a^n / factorial(n)) * P0;
    else
        P_theo(n+1) = (a^n / (factorial(c_val) * c_val^(n-c_val))) * P0;
    end
```

```
end

% Percent error for each P(N=n)
pct_err_P = 100 * abs(prob_empirical(1:length(P_theo)) - P_theo) ./
max(eps, abs(P_theo));

% Theoretical performance measures (standard M/M/c)
rho = lambda / (c_val * mu); % utilization per server
% Lq (E[number in queue]) using Erlang C formula:
% First compute Erlang C:
a = lambda / mu; % offered load (again)
ErlangC = (a^c_val / factorial(c_val) * (c_val / (c_val - a))) * P0;
Lq_theo = ErlangC * a / (c_val - a); % alternative form = ErlangC *
rho / (1-rho) * ??? avoid confusion
% More stable formula: Lq = ErlangC * a / (c*(1 - a/c))
Lq_theo = ErlangC * a / (c_val * (1 - a/c_val));

L_theo = Lq_theo + a;           % Little: L = Lq + a
W_theo = L_theo / lambda;       % mean sojourn
Wq_theo = Lq_theo / lambda;     % mean waiting

% Empirical estimates:
% mean sojourn (empirical)
W_emp = mean(sojourns);
% L_emp (time-average) from prob_empirical:
n_vals = 0:(length(prob_empirical)-1);
L_emp = sum(n_vals .* prob_empirical);
% Lq_emp: time-average of customers in queue (requires tracking queue
length over time).
% If you tracked queue-length time-weighted vector queue_time_in_state
with index q=0..Qmax:
% you can compute Lq_emp = sum(q_vals .* queue_time_prob).
% If not available, approximate Lq_emp = L_emp - (average number busy
servers).
% average busy servers = sum_{n} min(n,c)*P(N=n)
avg_busy = sum( min(n_vals, c_val) .* prob_empirical );
Lq_emp = L_emp - avg_busy;

Wq_emp = Lq_emp / lambda; % by Little
% Percent errors
pct_err_L = 100 * abs(L_emp - L_theo) / max(eps, abs(L_theo));
pct_err_Lq = 100 * abs(Lq_emp - Lq_theo) / max(eps, abs(Lq_theo));
pct_err_W = 100 * abs(W_emp - W_theo) / max(eps, abs(W_theo));
pct_err_Wq = 100 * abs(Wq_emp - Wq_theo) / max(eps, abs(Wq_theo));

% Display summary
fprintf('Metric\tTheoretical\tEmpirical\t%% error\n');
```

```
Metric    Theoretical    Empirical    % error
```

```
fprintf('L\t%.6f\t%.6f\t%.2f%%\n', L_theo, L_emp, pct_err_L);
```

```
L    0.450437    0.355488    21.08%
```

```
fprintf('Lq\t%.6f\t%.6f\t%.2f%%\n', Lq_theo, Lq_emp, pct_err_Lq);
```

```
Lq    0.020787    0.008936    57.01%
```

```
fprintf('W\t%.6f h\t%.6f h\t%.2f%%\n', W_theo, W_emp, pct_err_W);
```

```
W    0.033783 h    0.033684 h    0.29%
```

```
fprintf('Wq\t%.6f h\t%.6f h\t%.2f%%\n', Wq_theo, Wq_emp, pct_err_Wq);
```

```
Wq    0.001559 h    0.000670 h    57.01%
```

```

% Show top n probabilities differences (first 10 or available)
Nshow = min(10, length(P_theo));
fprintf('\nn\tP_theo\tP_emp\t%% err\n');
```

```
n    P_theo    P_emp    % err
```

```
for n=0:(Nshow-1)
    fprintf('%d\t%.5f\t%.5f\t%.2f%%\n', n, P_theo(n+1),
prob_empirical(n+1), pct_err_P(n+1));
end
```

```
0    0.64633    0.70511    9.10%
1    0.27769    0.24322    12.41%
2    0.05966    0.04273    28.37%
3    0.01282    0.00894    30.27%
4    0.00275    0.00000    100.00%
5    0.00059    0.00000    100.00%
6    0.00013    0.00000    100.00%
7    0.00003    0.00000    100.00%
8    0.00001    0.00000    100.00%
9    0.00000    0.00000    100.00%
```

Questions:

Part A:

1.

a) L: 5

b) L_q: 25/6

c) W: 1/2

d) W_q: 5/12

2.

The simulated results differ from theory by 6%, 10% and 4% respectively. The theoretical value lies within the simulations confidence interval for the first two so the discrepancy is consistent with sampling error but the last stat is not within the confidence interval so the discrepancy sugests a bias.

Part B:

1.

a)P_n : for n<c: ((lambda/mu)^n)/n! *P_0

for n => c: ((lambda/mu)^n)/ (c! * c^(n-c))*P_0

b) L: 12/15

c) L_q: 16/15

d)W: 3/50

e) W_q: 2/72

2.

Overall, the simulation and theory agree qualitatively, the most probable states where n= 0,1 are in the same order and contain most of the probability with a relative 9-12% error. in n= 2,3, there is a larger relative error 28-30%. for n =>4, there probabilites are super small so the 100% relative error is misleading